

Python Fundamentals (Assignment1)

Assignment Problems

Q1. Write a program that asks the user for their name and age, then prints a sentence like:

```
Hello Shradha, you are 21 years old!
```

Q2. Take two numbers as input from the user and print their sum, difference, product, and quotient.

Q3. Ask the user to enter two integers and one float. Convert them all to floats and print their average.

Q4. The user enters a string containing a number (e.g., "45"). Convert it to:

- an integer
- a float
- a string again

Print all three values with their types.

Q5. Evaluate and print the result of the following expression:

```
x = 10 + 3 * 2 ** 2
```

Based on what you learnt in the lecture explain why the output is what it is.

Q6. Write a program to swap values of two numbers entered by the user.

Q7. Ask the user for a temperature in Celsius (string input). Convert it to , then calculate and print temperature in Fahrenheit.

Conversion formula: $FahrenheitTemp = (CelsiusTemp * (9/5)) + 32$

Q8. Take the radius (r) as user input and print the area.

Use the formula: $Area = \pi * r^2$ (value of $\pi = 3.14$)

Q9. Ask the user for: Principal (P), Rate (R), Time (T). Convert all to and compute simple interest:

$$SI = (P * R * T) / 100$$

Q10. Take a decimal number as input (like 45.78) and output its:

- integer part - 45
- fractional part - .78

Python Fundamentals (Assignment2)

Note - To solve some of the assignment Qs we'll have to learn some additional logic (already mentioned as hints). The purpose of these questions is to help build programming logic so even if we have to take hint for mathematical part of the problem & we write the Python code on our own, it's alright.

Assignment Problems

Q1. Write a program that takes `salary` as input. Using conditional statements, calculate the **final tax rate** based on these rules:

- If salary < 30,000 → 5%
- If salary is 30,000–70,000 → 15%
- If salary > 70,000 → 25%

Q2. Write a function that takes two integers `a` and `b` and prints all even numbers between them (inclusive).

Q3. Write a function that prints the **digits** of a number, `n`.

For eg: `n = 312`, there are 3 digits in it 3, 1 and 2 & we need to print them.

[Hint - The right most digit of a number N is $N \% 10$.

And to remove the right most digit from a number, we can do $N = N / 10$.]

Q4. Write a function to return the count the number of digits in a number, `n`.

Q5. Write a function to return the **sum of digits** of a number, `n`.

Q6. Write a program to print all numbers from 1 to 100 that are divisible by both 3 and 5.

Q7. Design a program to continuously input a number from user & print if it is positive or negative until the user enters "Quit".

Q8. Let's create a Simple Calculator that performs arithmetic operations. Create a function `calculator(a, b, operation)` that performs addition, subtraction, multiplication, or division based on the `operation` parameter.

[`operation` parameter can have values `'+'`, `'-'`, `'*'` & `'/'` .

Q9. Write a function `s_prime(n)` that returns `true` if `n` is a prime number and `False` otherwise, using a loop.

[Hint -

1. We only check prime for 2 or numbers greater than 2. 2. is the smallest prime number.
2. A non-Prime number, `n`, will always get divided by atleast one number in range `[2, n-1]`.

Eg - For number 9 we'll check in range (2, 8) & it'll get divided by 3. So is non-prime & we'll return false for it.

For number 7 we'll check in range (2, 6) & it won't get divided by any. So 7 is prime & we'll return true for it.]

Q10. Let's create a "Number Guessing Game". Given a secret number (already decided by you), write a program that asks the user to guess it and prints:

- `"Too high"` if the guess is above the number
- `"Too low"` if the guess is below
- `"Correct!"` if the guess matches

Python Fundamentals (Assignment3)

Assignment Problems

Q1. Ask the user for a string and check whether it is a palindrome or not.

A **palindrome** is a string which is same when we read it forward & backward. Eg - "madam", "racecar" etc.

[**Hint** - A palindrome string is equal to the reversed version of the string. We can use a loop to reverse the string manually.]

Q2. Given a list of integers compute the average of all numbers in the list.

Q3. Input two lists of integers from the user. Merge them into one list and sort the result.

Eg - `list1 = [1, 2, 7]` , `list2 = [2, 4, 5]`

`result = [1, 2, 3, 5, 4, 5, 7]`

Q4. Given a tuple of integers, create:

- A tuple of all even numbers
- A tuple of all odd numbers

Q5. Create a dictionary where:

- Keys = student names
- Values = marks (integer)

Write a menu-based program where user presses a key ('A', 'B', 'C', 'D') depending on the operation they want to perform on the dictionary:

1. **A** - Add a student
2. **B** - Update marks
3. **C** - Search for a student
4. **D** - Display all students and marks

Q6. Given a list of words:

```
words = ["apple", "banana", "kiwi", "cherry", "mango"]
```

Create a dictionary that maps each word to its length.

Example:

```
{"apple": 5, "banana": 6, "kiwi": 4, ...}
```

Q7. Write a program that takes a string from the user and prints the number of spaces in the string.

Q8. Write a program to check whether two lists share no common elements.

```
# share no common elements list1 = [1, 2, 3, 4] list2 = [5, 6, 7, 8]  
# share common elements list1 = [1, 2, 3] list2 = [3, 4]
```

[Hint - use sets]

Q9. Given a list, print all elements that appear more than once in the list.

[Hint - use sets]

Q10. Ask the user for a string and print:

- All unique characters
- The count of unique characters

Python Fundamentals (Assignment4)

Assignment Problems

| Concept: Classes & Objects

Q1. Create a `BankAccount` class with attributes `account_number`, `owner_name`, and `balance`.
Add methods to `deposit`, `withdraw`, and `check balance`.

| Concept: Classes & Objects

Q2. Create a class `Book` with the following attributes:

- title
- author
- list of reviews

And add methods to:

- add a new review
- count reviews
- display all reviews

| Concept: Encapsulation

Q3. Create a class `Student` with **private** attributes `_name`, `_roll_no`, and `_marks`.
Provide **getter** and **setter** methods with validation (e.g., marks cannot be negative, roll number has to be between 1 & 100 & name cannot be empty).

| Concept: Function Overriding

Q4. Create a class `Shape` with a method `area()`.
Create subclasses `Circle`, `Rectangle`, and `Triangle` that override the `area()` method.

| Concept: Inheritance

Q5. Create a base class `Vehicle` with attributes like brand and model. Create two subclasses `Car` and `Bike` that add extra attributes - seats (in `Car`) & engine_cc (in `Bike`).

| Concept: Abstraction

Q6. Create an abstract class `Employee` with an abstract method `calculate_salary()`. Create subclasses `Intern`, `FullTimeEmployee`, and `ContractEmployee` that implement the method differently.

| Concept: Constructor Overloading (with Default Parameters)

Q7. Create a class `Person` that allows the constructor to work with:

- name only
- name + age
- name + age + address

As direct constructor overloading (multiple constructors) are not allowed but we have to use **default parameters** to simulate constructor overloading.

| Concept: Instance & Class Attributes

Q8. Create a class `Player` with:

- a class variable `player_count`
- instance variables `name` and `level`

Track how many players were created.

| Concept: Multiple Inheritance

Q9. Create the following classes: `Herbivore`, `Carnivore`, `Omnivore` with some attributes & methods. Then create a class `Bear` that inherits from all the above classes to showcase how multiple inheritance works.

| Concept: OOP

Q10. Mini Project - OOP Chat System

Let's create a Chat System using OOPs concepts. We have to create classes:

- User
- Message
- ChatRoom

And we have to implement functions:

- sending messages
- viewing chat history
- user joining and leaving the chatroom

Python Fundamentals (Assignment5)

Assignment Problems

Q1. Create a program that:

1. Opens a file *"names.txt"* in write mode
2. Writes 5 names (one per line) entered by the user
3. Then opens the same file in read mode and prints all names

Q2. Create a program that:

1. Opens a file *"log.txt"* in append mode
2. Adds a new log entry (like "Program run successfully")
3. Opens the file in read mode and prints all logs

Q3. Create a program that:

1. Has a list of numbers: `[5, 10, 15, 20, 25]`
2. Uses a list comprehension to create a new list with only numbers greater than 15
3. Prints the new list

Q4. Create a Python dictionary of 3 cities and their populations. Save it to *"cities.json"*.

1. Then load the JSON and print each city and its population.
2. Ask the user for a new city & its population - update this info in the json file.

Q5. Write a program that tries to open *"data.txt"* in read mode. If the file does not exist, catch the exception and print "File not found!".